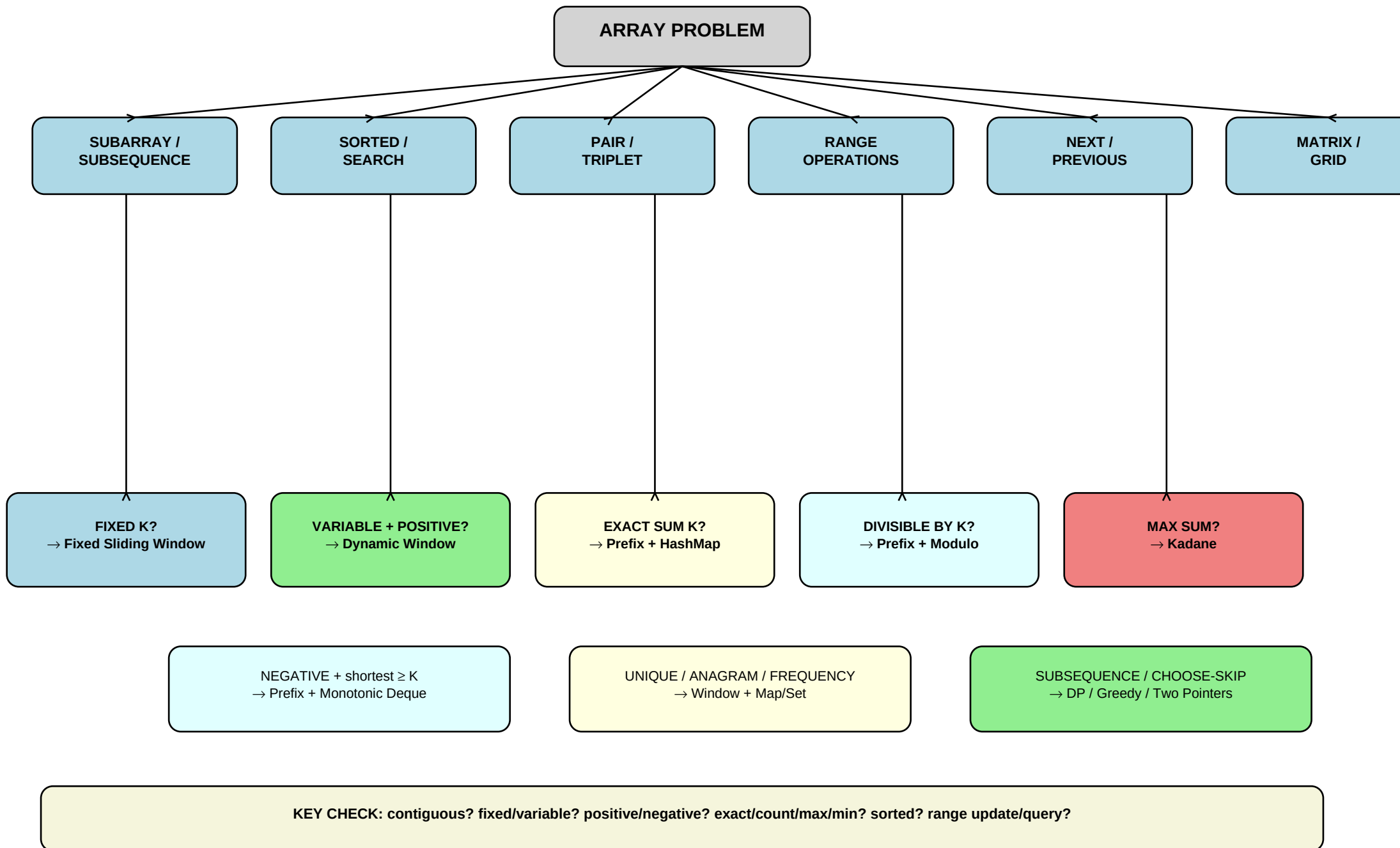
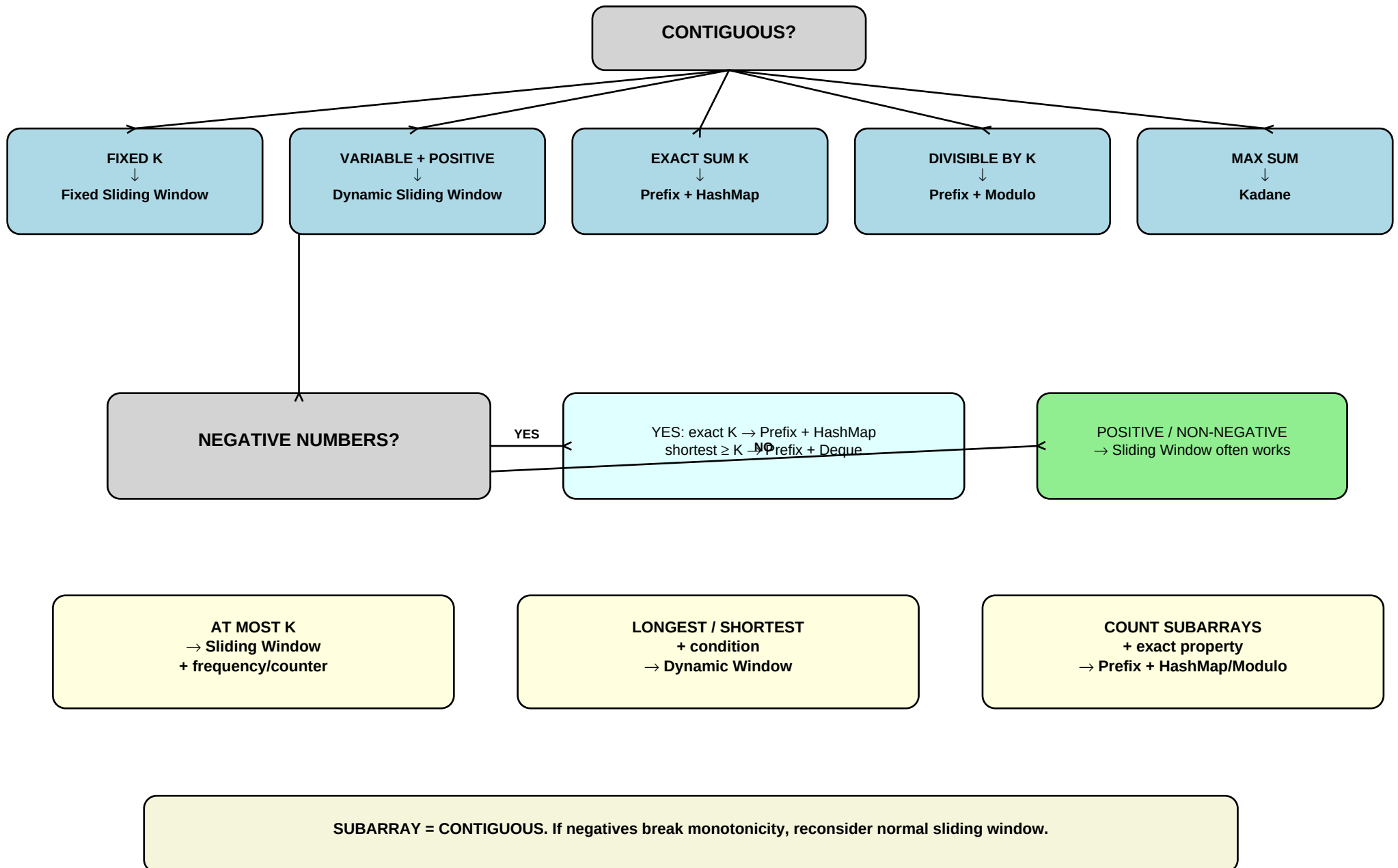


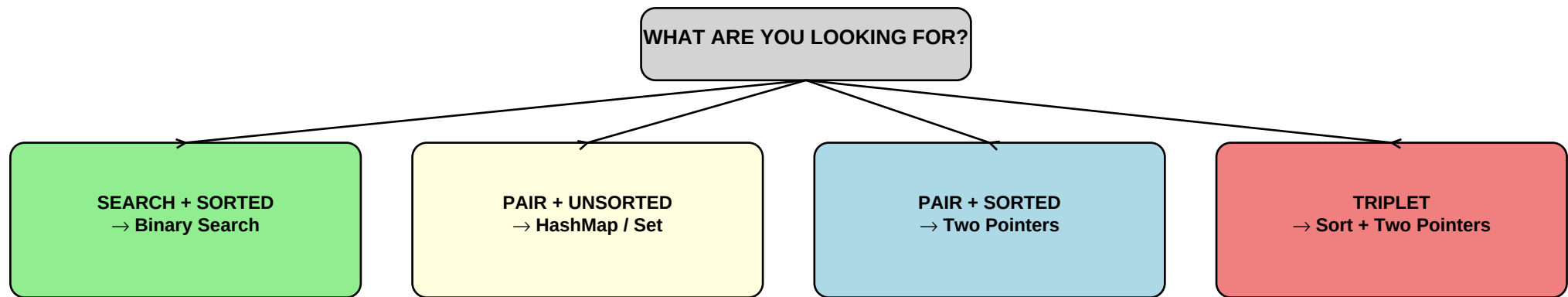
DSA ARRAY — MASTER PATTERN DECISION TREE



SUBARRAY DECISION TREE



SEARCH + PAIR + TRIPLET DECISION TREE



MIN/MAX ANSWER
+ candidate can be checked
+ monotonic feasibility
→ Binary Search on Answer

TWO POINTER SIGNALS
sorted • left/right • opposite ends
remove duplicates • partition • closest

HASHMAP
value→index
value→frequency
prefixSum→count/earliest index

ASK: "Do I need to remember something already seen?"
YES → HashMap / HashSet

SORTED ARRAY = USE THE ORDER. Search → Binary Search | Pair → Two Pointers.

RANGE QUERY + PREFIX SUM + DIFFERENCE ARRAY

RANGE / INTERVAL OPERATION?

RANGE SUM
Static / many queries
→ Prefix Sum

RANGE UPDATE
Add X to [L,R]
→ Difference Array

UPDATE + QUERY
→ Fenwick / Segment Tree

2D RECTANGLE
Sum → 2D Prefix
Update → 2D Difference

PREFIX + HASHMAP
exact sum / count / longest
negative values allowed

PREFIX + MODULO
sum divisible by K
same remainder → valid range

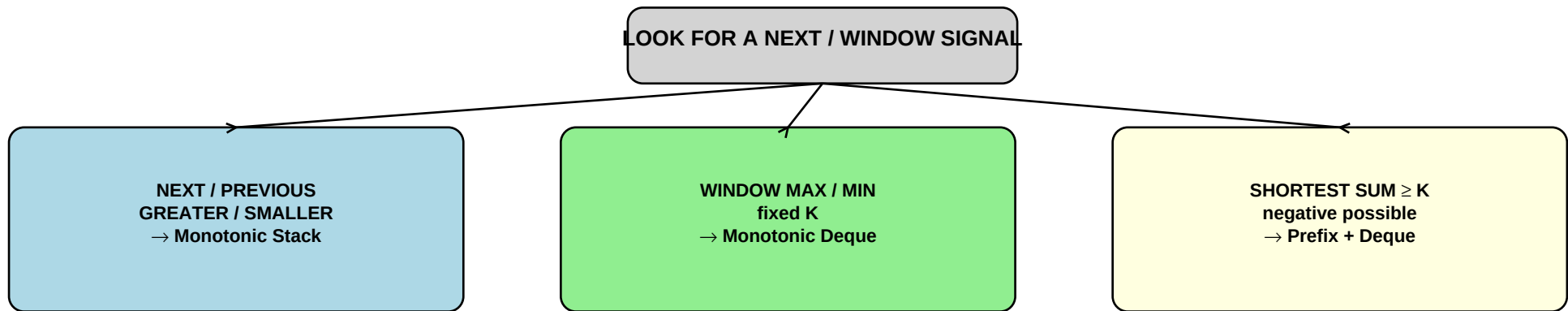
PREFIX XOR
range XOR

2D DIFFERENCE
rectangle update → four corners
then prefix reconstruction

SWEEP LINE
start +1, end -1
then prefix / sort events

MEMORY: PREFIX = answer range queries. DIFFERENCE = apply many range updates.

MONOTONIC STACK + DEQUE DECISION TREE



MONOTONIC STACK
while top is useless: pop()
then push current index/value

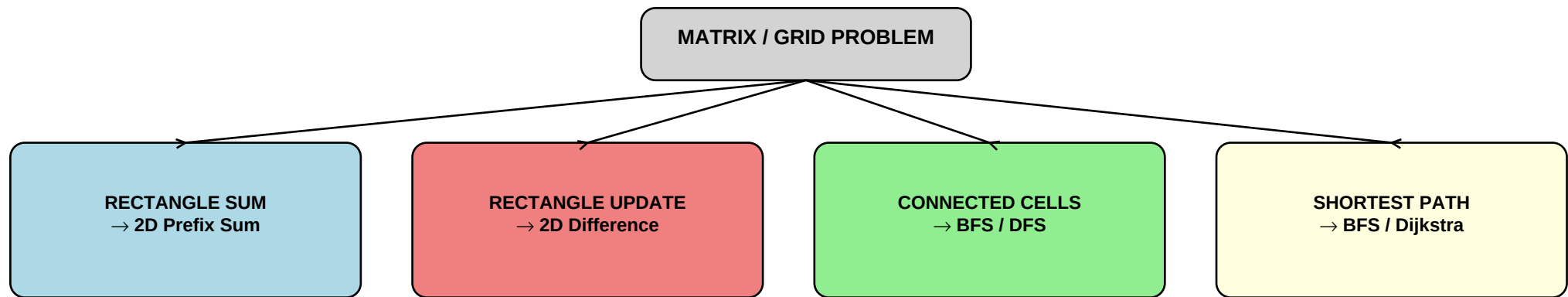
MONOTONIC DEQUE
remove worse from back
remove expired from front
front = current best

WORDS: next • previous • nearest • greater • smaller

WORDS: window • maximum • minimum • $O(n)$

STACK = nearest relationship. DEQUE = best candidate inside a moving window.

2D ARRAY / MATRIX DECISION TREE



SPIRAL / BOUNDARY / ROTATION
→ Simulation
(top, bottom, left, right)

ISLAND / REGION / COMPONENT
→ BFS / DFS
mark visited

GRID WEIGHTS
min/max path
→ DP / Dijkstra

ROW / COLUMN SUMS
→ Prefix / Suffix / Frequency

2D clue: rectangle → Prefix/Difference. Neighbors/connectivity → BFS/DFS.

ARRAY PATTERN — QUICK RECOGNITION MAP

Fixed K contiguous	Fixed Sliding Window
Longest/shortest + positive + condition	Dynamic Sliding Window
At most K / unique / frequency	Sliding Window + Map/Set
Exact subarray sum K	Prefix Sum + HashMap
Longest subarray sum K	Prefix Sum + earliest index
Count subarrays / divisible by K	Prefix + HashMap / Modulo
Maximum contiguous sum	Kadane
Negative + shortest sum $\geq K$	Prefix + Monotonic Deque
Sorted + search	Binary Search
Sorted + pair	Two Pointers
Pair + unsorted	HashMap
Triplet	Sort + Two Pointers
Next/previous greater/smaller	Monotonic Stack
Window maximum/minimum	Monotonic Deque
Range sum	Prefix Sum
Range update	Difference Array
Range update + query	Fenwick / Segment Tree
FAST RULE: Contiguous? → Window/Prefix/Kadane Sorted? → Binary Search/Two Pointers Range? → Prefix/Difference	
Top K	Heap

BEFORE CODING — 30 SECOND ARRAY CHECKLIST

1	SUBARRAY or SUBSEQUENCE?	Subarray = contiguous
2	SORTED?	Use the order if available
3	FIXED K?	Fixed Sliding Window
4	POSITIVE / NEGATIVE?	Positive often enables window; negatives often need prefix
5	EXACT / AT MOST / AT LEAST?	Exact → Prefix often; At most → Window often
6	SUM / COUNT / MAX / MIN?	Determines the core structure
7	PAIR / TRIPLET?	HashMap or Two Pointers
8	NEXT / PREVIOUS?	Monotonic Stack
9	RANGE SUM / UPDATE?	Prefix / Difference
10	CAN I CHECK AN ANSWER?	Binary Search on Answer

FINAL MINDSET

Don't memorize "Problem X = Pattern Y."
Memorize the SIGNAL that forces the pattern.